

Laboratorio 1 – Primeros desarrollos

Sesión 1: Algoritmo de Euclides para el MCD en Java

Adam El Bouchaibi Charrouf

October 4, 2026

Contents

1 Códigos Java

La numeración de líneas que aparece a la izquierda de cada listado es la que se utiliza en todas las tablas de este documento (la línea 1 es la declaración `package`).

1.1 Apartado 4.2: versión iterativa en main

Package: edu.upc.etsetb.poo.laboratorio.sesion1.apartado2

Clase: MaxComDivisorMainIterativo

```
1 package edu.upc.etsetb.poo.laboratorio.sesion1.apartado2;
2
3 public class MaxComDivisorMainIterativo {
4
5     public static void main(String[] args) {
6
7         int m = 8, n = 16, dividendo, divisor, resto;
8
9         if (m > n) {
10             dividendo = m;
11             divisor = n;
12         } else {
13             dividendo = n;
14             divisor = m;
15         }
16
17         resto = dividendo % divisor;
18
19         while (resto != 0) {
20             dividendo = divisor;
21             divisor = resto;
22             resto = dividendo % divisor;
23         }
24
25         System.out.println("El MCD es: " + divisor);
26     }
27 }
```

1.2 Apartado 4.3: versión iterativa con función

Package: edu.upc.etsetb.poo.laboratorio.sesion1.apartado3

Clase: MaxComDivisorFuncionIterativo

```
1 package edu.upc.etsetb.poo.laboratorio.sesion1.apartado3;
2
3 public class MaxComDivisorFuncionIterativo {
4
5     public static void main(String[] args) {
6
7         int m = 40, n = 16;
8
9         MaxComDivisorFuncionIterativo obj = new MaxComDivisorFuncionIterativo();
10        int res = obj.mcd(m, n);
11        System.out.println("El máximo común divisor entre " + m + " y " + n + " es: "
12            + res);
13    }
14
15    public int mcd(int dividendo, int divisor) {
16        int m = dividendo, n = divisor, resto;
17
18        if (m > n) {
19            dividendo = m;
20            divisor = n;
21        } else {
22            dividendo = n;
23            divisor = m;
24        }
25        resto = dividendo % divisor;
26        while (resto != 0) {
27            dividendo = divisor;
28            divisor = resto;
29            resto = dividendo % divisor;
30        }
31        return divisor;
32    }
33 }
```

```

21         dividendo = n;
22         divisor = m;
23     }
24
25     resto = dividendo % divisor;
26
27     while (resto != 0) {
28         dividendo = divisor;
29         divisor = resto;
30         resto = dividendo % divisor;
31     }
32
33     return divisor;
34 }
35 }

```

1.3 Apartado 4.4: versión recursiva

Package: edu.upc.etsetb.poo.laboratorio.sesion1.apartado4

Clase: MaxComDivisorFuncionRecursivo

```

1 package edu.upc.etsetb.poo.laboratorio.sesion1.apartado4;
2
3 public class MaxComDivisorFuncionRecursivo {
4
5     public static void main(String[] args) {
6
7         int m = 40, n = 16;
8
9         MaxComDivisorFuncionRecursivo obj = new MaxComDivisorFuncionRecursivo();
10        int res = obj.mcd(m, n);
11        System.out.println("El máximo común divisor entre " + m + " y " + n + " es: "
12            + res);
13    }
14
15    public int mcd(int dividendo, int divisor) {
16
17        if (divisor == 0) {
18            return dividendo;
19        } else {
20            return mcd(divisor, dividendo % divisor);
21        }
22    }
23 }

```

2 Cuestionario y tablas del laboratorio

2.1 Apartado 4.1: implementación en el main()

Las líneas citadas corresponden al listado de `MaxComDivisorMainIterativo` (apartado 4.2 del código).

1. ¿Cuándo no se ejecutará el bucle? El bucle no se ejecuta cuando el primer resto calculado ya es cero, es decir, cuando el divisor (el menor de los dos números) divide exactamente al dividendo (el mayor). Esto incluye el caso $m = n$. La condición de entrada al bucle es:

- Línea 19: `while (resto != 0){`

y el primer resto se calcula en:

- Línea 17: `resto = dividendo % divisor;`

2. ¿Qué valor deberá tomar entonces el resultado? El resultado es el propio divisor, es decir, el menor de los dos números, ya que no hay ninguna iteración que lo modifique. La línea que lo muestra es:

- Línea 25: `System.out.println("El MCD es: "+ divisor);`

Tabla de comprobación.

m	n	MCD resultante	¿Cuántas veces se ejecuta el bucle?
8	16	8	0
40	16	8	1
13	120	1	2
165	616	11	4

2.2 Apartado 4.2: depuración iterativa

El breakpoint se coloca dentro del bucle, en la línea que calcula el nuevo resto, de modo que el depurador se detiene una vez por iteración:

- Línea 22: `resto = dividendo % divisor;`

En la columna de restos sucesivos se incluye el primer resto (calculado en la línea 17, antes del bucle) y el último, que es 0.

m	n	Iteraciones	Restos sucesivos	Línea del breakpoint	Línea textual del breakpoint
8	16	0	0	22 (no se alcanza)	<code>resto = dividendo % divisor;</code>
40	16	1	8, 0	22	<code>resto = dividendo % divisor;</code>
13	120	2	3, 1, 0	22	<code>resto = dividendo % divisor;</code>
165	616	4	121, 44, 33, 11, 0	22	<code>resto = dividendo % divisor;</code>
64	1024	0	0	22 (no se alcanza)	<code>resto = dividendo % divisor;</code>
443	701	8	258, 185, 73, 39, 34, 5, 4, 1, 0	22	<code>resto = dividendo % divisor;</code>

Observación. En los pares (8, 16) y (64, 1024) el divisor divide al dividendo, por lo que el bucle no se ejecuta y el breakpoint de la línea 22 no llega a activarse. Para ver el único resto calculado en esos casos hay que colocar un breakpoint adicional en la línea 17 (`resto = dividendo % divisor;`, fuera del bucle).

MCD obtenido: (8, 16) $\rightarrow$ 8; (40, 16) $\rightarrow$ 8; (13, 120) $\rightarrow$ 1; (165, 616) $\rightarrow$ 11; (64, 1024) $\rightarrow$ 64; (443, 701) $\rightarrow$ 1.

2.3 Apartado 4.5: depuración de la función recursiva

Los breakpoints se colocan en las dos instrucciones `return` de `MaxComDivisorFuncionRecursivo`:

- Línea 17 (caso base): `return dividendo;`
- Línea 19 (llamada recursiva): `return mcd(divisor, dividendo % divisor);`

La columna *resto* es el segundo argumento de la invocación recursiva, `dividendo % divisor`. En la llamada que ejecuta el caso base no hay nueva invocación, por lo que se indica “–”.

Caso 1: $m = 8, n = 16$ (resultado: 8)

Invocación	dividendo	divisor	resto	Línea del <code>return</code>	Línea textual
1	8	16	8	19	<code>return mcd(divisor, dividendo % divisor);</code>
2	16	8	0	19	<code>return mcd(divisor, dividendo % divisor);</code>
3	8	0	–	17	<code>return dividendo;</code>

Caso 2: $m = 16, n = 40$ (resultado: 8)

Invocación	dividendo	divisor	resto	Línea del <code>return</code>	Línea textual
1	16	40	16	19	<code>return mcd(divisor, dividendo % divisor);</code>
2	40	16	8	19	<code>return mcd(divisor, dividendo % divisor);</code>
3	16	8	0	19	<code>return mcd(divisor, dividendo % divisor);</code>
4	8	0	–	17	<code>return dividendo;</code>

Caso 3: $m = 616, n = 165$ (resultado: 11)

Invocación	dividendo	divisor	resto	Línea del <code>return</code>	Línea textual
1	616	165	121	19	<code>return mcd(divisor, dividendo % divisor);</code>
2	165	121	44	19	<code>return mcd(divisor, dividendo % divisor);</code>
3	121	44	33	19	<code>return mcd(divisor, dividendo % divisor);</code>
4	44	33	11	19	<code>return mcd(divisor, dividendo % divisor);</code>
5	33	11	0	19	<code>return mcd(divisor, dividendo % divisor);</code>
6	11	0	–	17	<code>return dividendo;</code>

Nota. La versión recursiva entregada no intercambia los argumentos para que el dividendo sea el mayor. Por eso, cuando $m < n$ (casos 1 y 2), la primera invocación realiza una llamada extra que simplemente intercambia los valores (en el caso 1, `mcd(8, 16)` produce `mcd(16, 8)`). El resultado final es correcto en todos los casos.